Reliable Market-Data Distribution for a Networked Electronic Exchange

Design Document

Computer Networks (CSC-303) – **Group 11**

Date: 05 October 2026
Version: 0.1

1 Summary

This document presents the design of a reliable market-data distribution system for a networked electronic exchange. The system contains a central matching engine, limit order books, TCP order entry, UDP multicast market data, binary market-data messages, a market simulator, performance tests, and a PyQt6 visualisation interface.

The design addresses the possibility that UDP multicast packets may be lost, duplicated, or reordered. It introduces two equivalent market-data feeds, global sequence-number tracking, duplicate suppression, gap detection, TCP recovery, bounded event history, snapshot recovery, checkpoint verification, fault-injection experiments, recovery-fairness experiments, telemetry, and a reliability dashboard.

The most important design recommendations are:

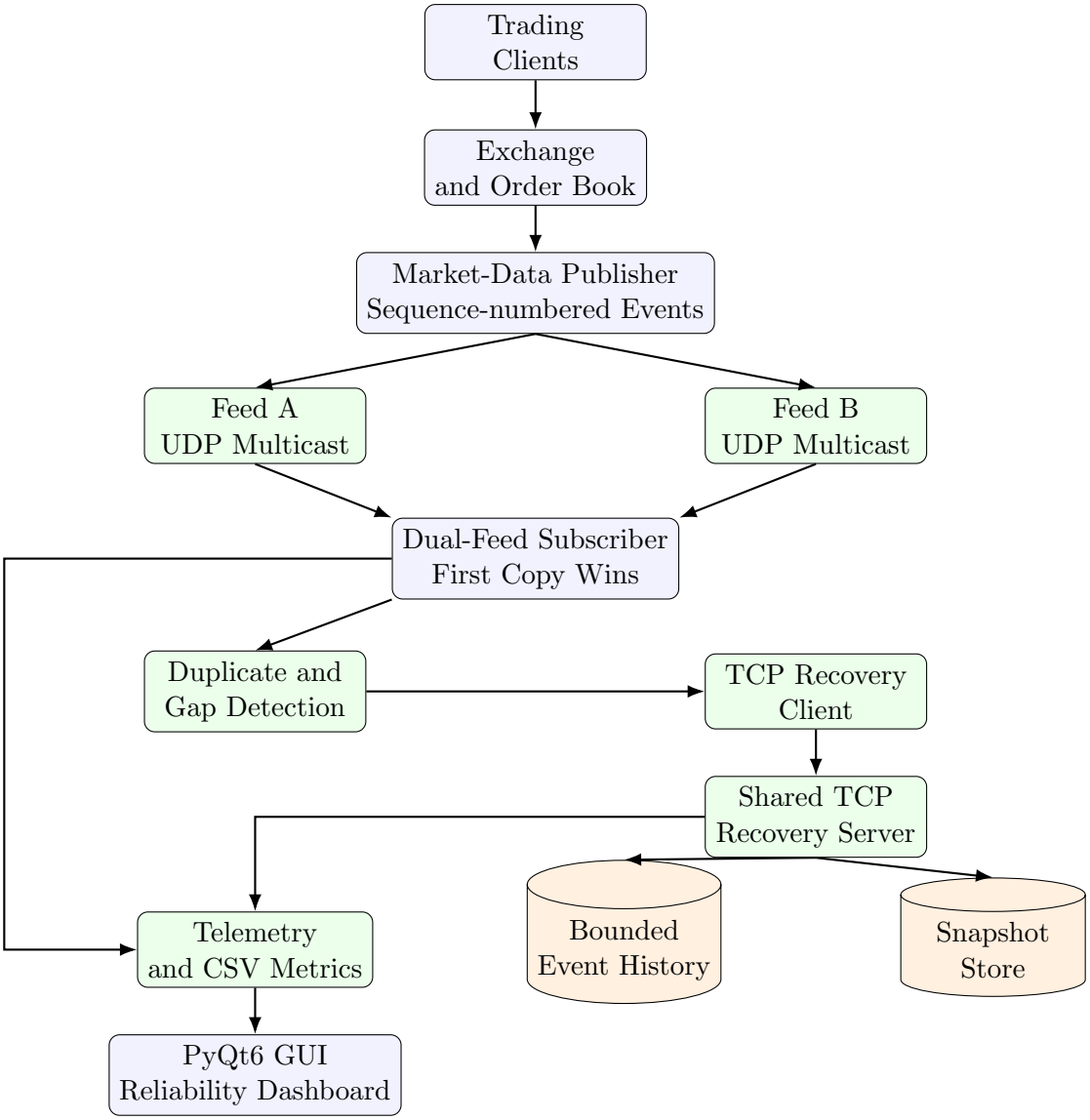
- Preserve the binary market-data protocol and use its message header sequence number, timestamp, message type, and instrument identifier.
- Publish the same sequence-numbered event on Feed A and Feed B. The subscriber uses a first-copy-wins rule.
- Use UDP for low-latency market-data distribution and TCP for recovery requests, replay responses, and snapshots.
- Maintain a configurable bounded in-memory event history. Requests outside retained history return `TOO_OLD` and trigger snapshot recovery.
- Use configurable snapshot intervals and retention counts; no fixed snapshot interval is assumed to be optimal.
- Compare FCFS and Round Robin scheduling at the shared TCP recovery server rather than introducing artificial subscriber delays.
- Use SHA-256 for final checkpoint hashing. The initial skeleton uses FNV-1a only as a deterministic non-cryptographic placeholder and it must be replaced before production implementation.

2 High-Level System Design

2.1 System Scope

The exchange is centralised. It does not use replicated order books, distributed consensus, primary-secondary exchange synchronisation, or distributed order-book replication.

The reliability components operate around the market-data publication and subscriber path.



2.2 System Components

Component	Role in the design
Market-data message header	Provides the sequence number, timestamp, message type, and instrument ID required by the reliability layer.
Market-data publisher and ring buffer	Produce ordered market-data events for publication to both feeds.
UDP multicast sender and receiver	Provide the transport integration points for independently configured Feed A and Feed B endpoints.
Matching engine and order books	Maintain the authoritative central market state and generate market-data events.
PyQt6 GUI	Displays market simulation data and provides a reliability and experimentation dashboard.

3 Program Design

3.1 Market-Data Event and Dual Feed

The reliability layer carries the established binary market-data event format.

```
struct MarketDataEvent {
    MessageHeader header;
    FeedId feed_id;
    vector<uint8_t> payload;
};
```

Both Feed A and Feed B carry the same event sequence:

$$E_i = \{\texttt{sequence} = i,\ \texttt{message},\ \texttt{timestamp},\ \texttt{instrument}\}$$

The feed identifier is transport metadata. It does not alter the sequence number or the meaning of the market-data event.

3.2 Sequence Tracking and Duplicate Detection

Each subscriber maintains one expected global sequence number.

```
onPacket(feedId, event):
    event.feed_id = feedId
    result = sequenceTracker.onEvent(event)

    if result == DUPLICATE:
        discard event
    else if result == GAP_DETECTED:
        notify recovery layer
        process event
    else:
        process event
```

For an event with sequence number s and expected sequence number e:

$$\text{result} = \begin{cases} \text{ACCEPTED}, & s = e \\ \text{DUPLICATE}, & s < e \\ \text{GAP_DETECTED}(e, s-1), & s > e \end{cases}$$

3.3 TCP Recovery Protocol

TCP is used after a subscriber detects a missing range.

```
RecoveryRequest {
    subscriber_id
    start_sequence
    end_sequence
}
```

```
RecoveryResponse {
    status: OK | TOO_OLD | BAD_REQUEST | ERROR
    recovered_events[]
    optional_snapshot
    detail
}
```

Recovery flow:

1. Subscriber detects missing range $[a, b]$.
2. Subscriber sends a `RecoveryRequest(a,b)` over TCP.
3. The recovery server checks bounded event history.
4. If all events are retained, the server returns `OK` and the recovered event sequence.
5. If the range is outside retained history, the server returns `TOO_OLD`.
6. The subscriber obtains the latest applicable snapshot, verifies it, applies it, and resumes from the snapshot sequence.

If multiple consecutive packets are missed, the subscriber does not send every sequence number; it sends only the range $[l, r]$, meaning packets l to r are missing, and those packets are then resent over the UDP connection.

3.4 Bounded Event History

`RecoveryHistory` stores only recent events:

$$H = \{E_{n-k+1}, E_{n-k+2}, \ldots, E_n\}$$

where k is the configurable maximum history size.
A request $[a, b]$ is recoverable only when:

$$a \geq \min(H) \quad \wedge \quad b \leq \max(H)$$

and every event in the requested sequence range is present. Otherwise, the response is `TOO_OLD`.

3.5 Snapshots and Checkpoint Verification

A snapshot represents the state of one instrument's order book at a sequence checkpoint.

```
OrderBookSnapshot {
    sequence_number
    instrument_id
    timestamp_ns
    book_state[]
    previous_hash
    hash
}
```

Snapshot configuration is experimental:

```
SnapshotConfig {
    enabled
    interval
    retention_count
}
```

The final checkpoint hash design is:

$$H_i = \text{SHA-256}(\text{CanonicalEncode}(S_i) \parallel H_{i-1})$$

where S_i is snapshot i, H_{i-1} is the previous snapshot hash, and the first snapshot uses an empty previous hash.

This provides tamper-evident checkpoint history. It does not create a tamper-proof database, distributed ledger, or consensus mechanism.

3.6 Recovery Fairness

The shared recovery server uses one of two scheduling policies:

```
enum class SchedulingPolicy {
    FCFS,
    ROUND_ROBIN
};
```

- **FCFS:** requests are served in arrival order.
- **Round Robin:** requests are grouped by subscriber ID and selected cyclically among subscribers with pending requests.

Fairness is evaluated using Jain's fairness index:

$$J = \frac{\left(\sum_{i=1}^{N} x_i\right)^2}{N \sum_{i=1}^{N} x_i^2}$$

where x_i is the number of recovery requests served for subscriber i.

3.7 Fault Injection and Experiments

`FaultInjector` uses a seeded pseudo-random generator. The same configuration and random seed must produce the same packet-loss decisions.

```
FaultConfig {
    packet_loss_rate
    duplication_rate
    reorder_rate
    delay
    random_seed
}
```

The principal experiment modes are:

1. Single-feed loss.

2. Dual-feed independent loss.
3. Dual-feed correlated loss.
4. Snapshot tampering.

In the snapshot tampering experiment, a stored snapshot is deliberately modified (for example, a changed book value, sequence number, or `previous_hash`) before recovery. The expected result is that checkpoint-chain verification fails, the subscriber rejects the snapshot, and the failure is reported on the History Integrity panel.

Parameter	Initial values	Purpose
Loss rate	0%, 0.1%, 1%, 5%	Measure reliability under loss
Duplication rate	0%, 1%	Validate duplicate suppression
Reordering rate	0% initially	Isolate packet-loss behaviour
Random seed	Fixed per experiment	Reproducibility
Scheduler	FCFS, Round Robin	Fairness comparison
Snapshot interval	Sweep values	Measure overhead/recovery trade-off
Retention count	Sweep values	Measure memory versus recoverability
Snapshot tampering	Off, on (modified field varied)	Validate checkpoint verification

3.8 Telemetry and Metrics

The following measurements are collected per run:

- throughput in events per second;
- packets/events sent and lost;
- duplicate count and gap count;
- recovery request count and recovered-event count;
- snapshot recoveries and unrecoverable gaps;
- recovery latency percentiles ($p50$, $p95$, $p99$, $p99.9$, maximum);
- per-subscriber recovery service distribution;
- redundancy effectiveness.

Recovery latency is:

$$L_{\text{recovery}} = t_{\text{recovery complete}} - t_{\text{gap detected}}$$

Redundancy effectiveness is:

$$R_{\text{effectiveness}} = \frac{\text{gaps avoided by second feed}}{\text{potential single-feed gaps}}$$

4 GUI Design

The PyQt6 market simulation interface includes a docked `ReliabilityDashboard` containing the following panels:

- **Live System:** Feed A status, Feed B status, sequence progress, duplicate count, gaps, and recovery state.
- **Experiment Control:** loss rate, random seed, scheduling policy, snapshot interval, retention count, and experiment controls.

- **Redundancy:** single-feed versus dual-feed mode and gaps avoided by the second feed.
- **Snapshot:** checkpoint state, interval, retention, and latest checkpoint information.
- **Fairness:** FCFS versus Round Robin selection, waiting time, and service distribution.
- **Metrics:** throughput, loss, duplicates, gaps, and recovery latency percentile displays.
- **History Integrity:** latest checkpoint hash and chain verification result.

5 Application of the Project

This project is applicable to electronic trading systems where market data must reach many subscribers with low latency and high reliability. It is useful for stock exchanges, trading simulations, broker platforms, and financial-market research environments. The system reduces the effect of UDP packet loss by using redundant feeds, sequence-number checking, TCP recovery, and snapshot-based recovery. It also provides a controlled environment to compare reliability, recovery latency, redundancy effectiveness, and fairness between subscribers.

6 References

References

[1] J. Postel, *User Datagram Protocol*, RFC 768, August 1980. https://www.rfc-editor.org/rfc/rfc768

[2] J. Postel, *Transmission Control Protocol*, RFC 793, September 1981. https://www.rfc-editor.org/rfc/rfc793

[3] National Institute of Standards and Technology, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, August 2015. https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[4] R. Jain, D. Chiu, and W. Hawe, *A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems*, DEC Research Report TR-301, 1984.